

科目：データベース演習

第12回：SQL応用：サブクエリとウィンドウ関数

講師：鈴木源吾@gg_szk

前回の復習

1. 外部ジョインで欠落のあるデータを扱う
2. 一歩進んだジョイン
3. 組合せを生成するジョインでバスケット分析

今回のトピック

1. サブクエリ: 問合せの結果を問合せで利用する
2. ウィンドウ関数でグループ全体を対象にした計算をする

その他の資料：第8章

topic 1

サブクエリ: 問合せの結果を問合せで利用する

サブクエリとは？

あるSQL文1の検索結果を用いて、さらにSQL文2による検索をしたいときがある。それをプログラムを組まずに、SQL文だけで実現するために、SQL文2の中に、SQL文1をネストして（入れ子で）記述する問合せ（クエリ）を書くことができる。このような問合せをサブクエリ（副問合せ, sub query）と呼ぶ

SQL文の中の様々な箇所にSQL文を入れることが可能

- WHERE句でネストするサブクエリ
- SELECT句でネストするサブクエリ
- FROM句でネストするサブクエリ

例

最もよく使われるのは、WHERE句でネストするサブクエリである。特に、IN句との組合せで使われる。次のような例を考えてみる

- アクセスログの中で、商品に関わるページへのアクセスを求めたいとする
 - アクセスログは、access_logテーブルの行なので、どのページにアクセスしたかは、access_logテーブルのrequest_pathカラムでわかる
 - ページ（request_path）に関する情報はweb_pagesテーブルの行なので、そのページが商品に関わるかどうかは、web_pagesテーブルのpage_categoryカラムが値'item'をとることでわかる
 - access_logテーブルのrequest_pathカラムと、web_pagesテーブルのrequest_pathカラムを関連づけて調べればよい

OR句を使った解決

- web_pagesテーブルを参照すると、page_categoryカラムが値'item'をとるのは、request_pathカラムが'/items/1'のように'/items/商品番号'という値をとる場合である
- よって、access_logテーブルに対し、request_pathがこのような値を持つ条件をすべてORでつなげて検索すれば、ほしい商品に関わるページを情報を求められる
- 例えば、商品が2つだけであれば、2つの条件をORでつなげて、以下のようなSQL文で検索すればよい

```
SELECT * FROM access_log  
WHERE request_path = '/items/1' OR request_path = '/items/2'
```

- しかし、すべての条件をORでつなげるのは大変である

LIKEを使った解決

- 値の種類の多さに対応するには、**この例に限って言えば**、値の特徴に着目して、LIKE句を用いて解決することも可能である
- 商品に関するrequest_pathは、/items/1などの文字列であるから、/itemで始まる特徴を持つ
- LIKE句は、文字列のパターンマッチを行える。このようなパターンを求めるには、以下のようなSQL文にすればよい

```
SELECT * FROM access_log  
WHERE request_path LIKE '/items%';
```

- しかし、この方法は文字列の特徴に頼っているので一般的には使えない

IN句を使った解決

- 多くのOR句を使わずに簡易に条件を書く方法として、IN句がある
- 例えば、前ページのSQL文をIN句を利用すると以下のように書ける
- IN句の中のどれかの値に等しくなれば、条件が真となる

```
SELECT * FROM access_log  
WHERE request_path IN ('/items/1', '/items/2');
```

- IN句の()の中にすべての値を列挙すればよい、しかし、値の種類が多いと列挙は大変である

WHERE句でネストするサブクエリによる解決

- そこでサブクエリを利用して、IN句の () 中に「値そのもの」を入れるのではなく、「値を求めるSQL文」を入れることによって、欲しい情報の条件判定を行う
- まず、() 内の値をweb_pagesテーブルから求める問合せは以下となる

```
SELECT request_path FROM web_pages WHERE page_category = 'item';
```

- これをIN句の () 内に挿入すると以下のサブクエリが得られる。これにより、1つのSQL文で求める情報が検索できる

```
SELECT * FROM access_log  
WHERE request_path IN (  
    SELECT request_path FROM web_pages  
    WHERE page_category = 'item'  
);
```

WHERE句でネストするサブクエリ

WHERE句の条件とするカラムと、サブクエリの検索結果となるカラムは、「同じ」情報を扱うカラムでなくてはならない

- この場合は、URLのパス（の一部）を表す文字列
- データベースの用語で言うと、ドメインが一致している必要がある
 - （少なくとも）型が同じで、同じかどうかを比較できる必要がある

WHERE句の条件とするカラム

```
1 SELECT * FROM access_log
2 WHERE request_path IN (
3     SELECT request_path FROM web_pages
4     WHERE page_category = 'item'
5 );
```

サブクエリでの検索結果になるカラム

WHERE句でネストするサブクエリのイメージ

```
1 SELECT * FROM access_log
2 WHERE request_path IN (
3     SELECT request_path FROM web_pages
4     WHERE page_category = 'item'
5 );
```

このSQLの結果が,
'/items/1', '/items/2', '/items/3', ...

request_pathが上記のSQLの結果の
どれかと等しくなっている

データ出力 実行計画 メッセージ 通知

	request_time timestamp without time zone	request_method text	request_path text	customer_id integer	search_hit integer	referer text
1	2014-02-10 19:47:17	GET	/items/220	54386	[null]	[null]
2	2014-02-10 19:49:13	GET	/items/76	[null]	[null]	[null]
3	2014-02-10 19:54:55	GET	/items/27	[null]	[null]	/search
4	2014-02-10 19:52:26	GET	/items/317	11396	[null]	/search
5	2014-02-10 19:51:08	GET	/items/244	[null]	[null]	[null]

WHERE句でネストするサブクエリとジョイン

実は、今述べたサブクエリはジョインを使って書き換えることが可能である。
以下のジョインを使った問合せは、前ページのサブクエリを使った問合せと同じ検索結果が得られる。

```
SELECT * FROM access_log AS a
JOIN web_pages AS p ON a.request_path = p.request_path
WHERE p.page_category = 'item';
```

（個人の感覚にもよるが）サブクエリは、ジョインよりも直感的にわかりやすいと思われる。積極的に用いてかまわない（PostgreSQLでは性能は変わらない。過去には、DBMSによってはサブクエリの性能が低いこともあった（MySQLなど））。

SELECT句でネストするサブクエリ

ordersテーブルを用いて、注文の個数であるorder_amountカラムの平均を求めよう

```
SELECT AVG(order_amount) FROM orders;
```

→ 8275.96...という値が得られる

SELECT句でネストするサブクエリ

次に、顧客毎の注文個数の平均を求めよう。顧客IDであるcustomer_idによってグループ化すればよい。顧客IDの値も一緒に表示する

```
SELECT customer_id, AVG(order_amount) FROM orders  
GROUP BY customer_id;
```

	customer_id integer	avg numeric
1	79030	3625.000000000000000000
2	48993	5743.333333333333333333
3	33711	8302.500000000000000000
4	47051	4420.000000000000000000
5	91786	20040.0000000000000000

SELECT句でネストするサブクエリ

ここで、前ページの顧客毎の注文個数の平均の横に、前前ページで求めた全体での注文個数の平均を表示してみたい

- どの顧客が平均を超えているかを知りたい

つまり次のような結果を得たい

	customer_id integer	avg numeric	顧客毎の平均注文個数	avg numeric	全体の平均注文個数
1	79030		3625.0000000000000000		8275.9677289066602044
2	48993		5743.3333333333333333		8275.9677289066602044
3	33711		8302.5000000000000000		8275.9677289066602044
4	47051		4420.0000000000000000		8275.9677289066602044
5	91786		20040.0000000000000000		8275.9677289066602044

SELECT句でネストするサブクエリ

前ページの結果を得るために、SELECT句でネストするサブクエリが使える。
前に出てきた2つのSQL文を組み合わせれば良い。

サブクエリは()で囲む必要がある。()の結果が一つのカラムとして置き換えられる

- 注意：()の中で複数のカラムを返却するとエラーとなる（次ページ参照）

```
1 SELECT customer_id, AVG(order_amount), (SELECT AVG(order_amount) FROM orders)
2 FROM orders GROUP BY customer_id;
```

	customer_id integer	avg numeric	avg numeric
1	79030	3625.0000000000000000	8275.9677289066602044
2	48993	5743.3333333333333333	8275.9677289066602044
3	33711	8302.5000000000000000	8275.9677289066602044
4	47051	4420.0000000000000000	8275.9677289066602044
5	91786	20040.0000000000000000	8275.9677289066602044

サブクエリの結果が出ている

このSQLの結果が
8275.9677...

SELECT句でネストするサブクエリの制限：カラム数

- ()の中で複数のカラムを指定すると、以下のようにエラーとなる
- もし、この例にある、AVG(order_amount), MAX(order_amount) の2つを横に並べたかったら、それぞれをサブクエリとして、()を2つ並べれば良い

```
1 SELECT customer_id, AVG(order_amount), (SELECT AVG(order_amount), MAX(order_amount) FROM orders)
2 FROM orders
3 GROUP BY customer_id;
```

データ出力 実行計画 メッセージ 通知

```
ERROR: subquery must return only one column
LINE 1: SELECT customer_id, AVG(order_amount), (SELECT AVG(order_amo...
                                                ^
```

SQL 状態: 42601

文字: 40

SELECT句でネストするサブクエリの制限：行数

前の例では、サブクエリが1つの行のみ返していたが、サブクエリが複数行返すような場合はどうなるだろうか？AVG(order_amount)の代わりに、order_idを返すようにサブクエリに変更し実行すると、以下のように、1行より多い行が返ることでエラーとなる。都合よく直積をとってはくれない。

```
1 SELECT customer_id, AVG(order_amount), (SELECT order_id FROM orders) FROM orders
2 GROUP BY customer_id;
```

データ出力 実行計画 メッセージ 通知

ERROR: more than one row returned by a subquery used as an expression
SQL 状態: 21000

例題1

(1) ordersテーブルには、顧客のIDを表すcustomer_idカラムがある。また、customersテーブルにもcustomer_idカラムはあるが、さらに、顧客が住む都道府県を表すcustomer_locationカラムがある。

そこで、サブクエリを利用し、新潟県に住んでいる顧客の注文をordersテーブルから求めるSQL文を作成せよ。ただし、結果の行数は10に限定せよ。検索結果例（一部）を以下に示す

	 order_id [PK] integer	 order_time timestamp without time zone	 shop_id integer	 customer_id integer	 order_amount integer
1	398	2012-01-29 23:39:08	21	60561	2400
2	426	2012-01-30 21:21:23	8	7429	9900
3	427	2012-01-30 21:21:23	29	7429	3880
4	428	2012-01-30 21:21:23	6	7429	24570
5	563	2012-02-08 19:49:45	9	98815	6660

例題1

(2) ordersテーブルを用いて、顧客のID (customer_id)、顧客毎の注文個数の最大値、全体での注文個数の最大値の組を表示するSQL文を作成せよ。ただし、結果の行数は10に限定せよ。結果例（一部）を以下に示す

	 customer_id integer 	max integer 	max integer 
1	79030	5970	90270
2	48993	8520	90270
3	33711	14520	90270
4	68864	5510	90270
5	47051	4420	90270

演習：課題6-1

(1) ordersテーブルには、顧客のIDを表すcustomer_idカラムがある。また、customersテーブルにもcustomer_idカラムはあるが、さらに、顧客の年齢を表すcustomer_ageカラムがある。

そこで、サブクエリを利用し、年齢が30歳より低い（30歳は含まない）顧客の注文をordersテーブルから求めるSQL文を作成せよ。ただし、結果の行数は10に限定せよ。結果例（一部）を以下に示す

	 order_id [PK] integer	 order_time timestamp without time zone	 shop_id integer	 customer_id integer	 order_amount integer
1	5	2012-01-01 16:06:45	30	62709	7420
2	6	2012-01-01 16:06:45	6	62709	8270
3	7	2012-01-01 20:09:57	26	26209	18010
4	8	2012-01-01 20:09:57	30	26209	2100
5	12	2012-01-01 21:47:15	27	58394	9970

演習：課題6-1

(2) itemsテーブルは、商品に関する情報のテーブルである、商品を買っているショップのID (shop_id), 商品の価格 (item_price) というカラムを持っている.

そこで、ショップのID, ショップ毎の商品の価格の平均値, 全体での商品の価格の平均値の組を表示するSQL文を作成せよ. ただし, 結果の行数は10に限定せよ. 結果例 (一部) を以下に示す.

	shop_id integer	avg numeric	avg numeric
1	29	7389.0476190476190476	7761.6795865633074935
2	4	6721.8181818181818182	7761.6795865633074935
3	7	5126.0000000000000000	7761.6795865633074935
4	10	2314.0000000000000000	7761.6795865633074935
5	9	7544.0000000000000000	7761.6795865633074935

topic 2

ウィンドウ関数でグループ全体を対象にした計算をする

ウィンドウ関数とは？：GROUP BY句をおさらい

ウィンドウ関数はSQLの中では、比較的新しい機能であり、GROUP BY句を使いやすくしたいという動機で開発された

まずは、GROUP BY句をおさらいしよう。monthly_salesテーブルを例に用いる
monthly_salesテーブルは、月とショップ毎の売上個数のカラムを持つ（下表）

	月	ショップのID	売上個数
	sales_month date	shop_id integer	sales_amount integer
1	2014-01-01	10001	2351935
2	2014-01-01	10002	1510956
3	2014-01-01	10003	1913598
4	2014-01-01	10004	2323593
5	2014-02-01	10001	2035986
6	2014-02-01	10002	2029369

2014年1月には、IDが10001
のショップで2351935個の
商品が売れた

データの数は全部で24個
（左は一部）

ウィンドウ関数とは？：GROUP BY句をおさらい

ここで、月毎の売上個数の総数（つまり、すべてのショップの売上個数の総和）を求めてみよう。つまり求めたい結果は以下である

	sales_month date	sum bigint
1	2014-01-01	9195344
2	2014-02-01	9793312
3	2014-03-01	9789606
4	2014-04-01	1282091
5	2014-05-01	1362396
6	2014-06-01	1363277

2014年1月には、全体で9195344
個の商品が売れた

データの数全部で12個
(左は一部)

ウィンドウ関数とは？：GROUP BY句をおさらい

このように、カラムの値毎にグループ化して合計や平均を求めるのがGROUP BY句だった（第7回 topic 2）。以下のSQL文で前ページの結果が得られる。

（結果を見やすくするために、月の値でソートした→ORDER BY句）

グループを表す値（月）

グループ内の合計値

```
SELECT sales_month, SUM(sales_amount) FROM monthly_sales  
GROUP BY sales_month  
ORDER BY sales_month;
```

月を単位にグループを作る

ウィンドウ関数とは？：GROUP BY句に足りないこと

GROUP BY句は集計に大変便利だが、以下の点が物足りない

- 集計された結果だけに行が限定されてしまう

→ 集計はしたいが、「元の行も残したい」ニーズがある！

例) 月・店毎の売上個数と、月毎の全ての店の売上個数を見比べたい (下表)

元のテーブル

	sales_month date	shop_id integer	sales_amount integer
1	2014-01-01	10001	2351935
2	2014-01-01	10002	1510956
3	2014-01-01	10003	1913598
4	2014-01-01	10004	2323593
5	2014-02-01	10001	2035986
6	2014-02-01	10002	2029368



GROUP BYの集計結果を一緒に表示 (赤枠)

	sales_month date	shop_id integer	sales_amount integer	sum bigint
1	2014-01-01	10001	2351935	9195344
2	2014-01-01	10002	1510956	9195344
3	2014-01-01	10003	1913598	9195344
4	2014-01-01	10004	2323593	9195344
5	2014-02-01	10001	2035986	9195344
6	2014-02-01	10002	2029368	9793312

月・店毎の売上個数

月毎売上個数の総数

ウィンドウ関数の利用例

以下のSQL文で、前ページ右の検索結果を得られる。

このSUM()がウィンドウ関数である。これまで使ってきたSUM()と関数の名前は同じである。

```
SELECT sales_month, shop_id, sales_amount,  
       SUM(sales_amount) OVER (  
           PARTITION BY sales_month  
       )  
FROM monthly_sales  
ORDER BY sales_month;
```

SUM()の後にある、「OVER()」がウィンドウ関数を使うサインであり、ウィンドウ関数として使う場合は、必ずこのOVER()をつける必要がある

ウィンドウ関数の解説

```
SELECT sales_month, shop_id, sales_amount,  
       SUM(sales_amount) OVER (  
         PARTITION BY sales_month  
       )  
FROM monthly_sales  
ORDER BY sales_month;
```

この部分が1つの新しいカラムを表している

OVERの中に、ウィンドウ関数として必要な指定を入れていく

PARTITION BYが、「GROUP BY」と同等の機能。ここではsales_monthの値毎に区切り（パーティション）を作る＝グループを作ると同等

例えば、rank() ウィンドウ関数は、OVERの中にORDER BY句を記述すると、パーティション内でソートされた順番の数字を得ることができる

さらに売上の比率を求める

月毎のショップの売上個数の全体に対する比率を以下のように出せる

```
SELECT sales_month, shop_id, sales_amount,  
       CAST(sales_amount AS real)/SUM(sales_amount) OVER (  
         PARTITION BY sales_month  
       ) AS sales_ratio  
FROM monthly_sales  
ORDER BY sales_month;
```

sales_amountと、月の単位で集計した
sales_amountの比率を求めている

CASTが使われている理由は、
ないと整数の割り算になり、
答えが整数（つまり1と0）
になってしまうため

	sales_month date	shop_id integer	sales_amount integer	sales_ratio double precision
1	2014-01-01	10001	2351935	0.25577455286066514
2	2014-01-01	10002	1510956	0.16431750677299295
3	2014-01-01	10003	1913598	0.20810510188634596
4	2014-01-01	10004	2323593	0.2526923408194408
5	2014-01-01	10082	1095262	0.11911049766055516
6	2014-02-01	10002	2029368	0.20721978427727003

様々なウィンドウ関数

PostgreSQLでは以下のウィンドウ関数を利用することができる

- count : パーティション内の行数を数える
- sum : パーティション内の合計や累積和をとる
- avg : パーティション内の平均をとる
- rank : パーティション内をソートして順位を付ける (重複あり)
- row_number : パーティション内をソートして順位を付ける (重複なし)
- lag : パーティション内をソートして前の行の値をとる
- lead : パーティション内をソートして後の行の値をとる

例題2

(3) itemsテーブルは、商品を管理しているテーブルで、商品のID (item_id), ショップのID (shop_id), 商品の名前 (item_name), 商品の価格 (item_price) をカラムとして持っている。ウィンドウ関数AVGを用いて、商品のID, ショップのID, 商品の価格, ショップ内における商品価格の平均, の組を求めるSQL文を作成せよ。検索結果は最初の10件に限定せよ。以下に結果例（一部）を示す。

	 item_id [PK] integer 	shop_id integer 	item_price integer 	avg numeric 
1	371	1	5910	9078.0000000000000000
2	137	1	15490	9078.0000000000000000
3	363	1	7530	9078.0000000000000000
4	15	1	7820	9078.0000000000000000
5	314	1	8640	9078.0000000000000000
6	331	2	6830	10051.81818181818182

演習 課題6-2

(3) customersテーブルは、顧客を管理しているテーブルで、顧客のID (customer_id), 顧客の年齢 (customer_age), 顧客の住む場所 (都道府県) (customer_location) 等をカラムとして持っている。ウィンドウ関数AVGを用いて、顧客のID, 顧客の住む場所, 顧客の年齢, 同じ場所 (都道府県) に住む顧客の年齢の平均, の組を求めるSQL文を作成せよ。検索結果は最初の10件に限定せよ。以下に結果例 (一部) を示す。

	customer_id [PK] integer	customer_location text	customer_age integer	avg numeric
1	79641	三重県	12	28.5119565217391304
2	39760	三重県	32	28.5119565217391304
3	22182	三重県	51	28.5119565217391304
4	14382	三重県	63	28.5119565217391304
5	21194	三重県	68	28.5119565217391304
6	39227	三重県	25	28.5119565217391304